

Contents

Playwright Interview Questions & Answers — Complete Guide (2026 Edition)	1
1. Playwright Fundamentals	1
2. Locators & Selectors	2
3. Waiting Strategies & Flaky Tests	2
4. Actions & UI Interactions	3
5. Browser Contexts, Sessions & Authentication	4
6. API Testing & Network Interception	4
7. Test Organization: POM, Fixtures & Data-Driven Testing	5
8. Parallel Execution, Sharding & CI/CD	6
9. CLI Commands & Test Execution Reference	6
10. Debugging, Reporting & Tracing	7
11. Visual Regression Testing	8
12. Advanced / Often-Missed Features	8
13. Scenario-Based / Real-World Questions	9
14. General QA & SDLC Concepts	9
15. Best Practices & Common Pitfalls	10
16. Suggested Study Approach	10

Playwright Interview Questions & Answers — Complete Guide (2026 Edition)

A single, de-duplicated, topic-organized reference built from your four source PDFs, with gaps filled in so it covers beginner → advanced → real-world scenario questions, plus a CLI/framework-setup quick reference.

1. Playwright Fundamentals

Q1. What is Playwright? An open-source, Microsoft-built end-to-end testing/automation framework that drives Chromium, Firefox, and WebKit through a single API, designed for modern web apps (SPAs, PWAs).

Q2. Why is Playwright popular in 2026? Built-in auto-waiting, native parallel execution, cross-browser support with no separate driver binaries, strong CI/CD integration, network interception/mocking, mobile emulation, and built-in tracing/video/screenshot capture.

Q3. What browsers does Playwright support? Chromium (Chrome/Edge), Firefox, and WebKit (Safari engine) — all through one API.

Q4. How do you install Playwright?

```
npm init playwright@latest
```

This scaffolds config, installs browser binaries, and adds example tests.

Q5. What does `npm init playwright@latest` actually generate, and what does each piece do? | File/Folder | Purpose | `node_modules` | Installed project dependencies | `tests/` | Example/generated tests to validate the setup | `.gitignore` | Files/folders excluded from Git commits | `package.json` | Scripts and dependency metadata | `package-lock.json` | Locks dependency versions for reproducible installs | `playwright.config.ts` | The framework's primary configuration file (projects, timeouts, reporters, retries, etc.) |

Q6. Difference between Playwright and Selenium? | Aspect | Playwright | Selenium |
|—|—|—| | Waiting | Built-in auto-wait | Mostly manual/explicit waits | | Drivers | No separate driver binaries | Needs WebDriver per browser | | Parallelism | Native, isolated browser contexts | Requires extra setup (Grid) | | Modern web support | Strong (Shadow DOM, SPA) | Weaker out of the box | | Debugging | Trace Viewer, video, screenshots built-in | Limited natively |

Q7. What is headless mode? Running the browser without a visible UI — faster and typical for CI; toggle with `headless: false` to watch the browser during debugging.

2. Locators & Selectors

Q8. What is a locator in Playwright? An object that identifies one or more elements and re-queries the DOM every time an action or assertion runs against it (lazy evaluation), enabling auto-wait and retry.

Q9. What locator strategies are available?

```
page.locator('#id'); // CSS
page.getByRole('button', { name: 'Submit' }); // role (recommended)
page.getByText('Welcome'); // visible text
page.getByLabel('Email address'); // form label
page.getByPlaceholder('Enter your email');
page.getByAltText('Logo');
page.getByTitle('Close dialog');
page.getByTestId('submit-btn');
```

Best practice: prefer user-facing attributes (`getByRole`, `getByLabel`, `getByText`) over CSS/XPath, since they mirror how real users find elements and survive markup changes.

Q10. Difference between `page.locator()` and `page.$()` / `page.$$()`? `locator()` is lazily evaluated, auto-waits, retries, and works with web-first assertions. `page.$()`/`page.$$()` return an `ElementHandle` immediately with no auto-wait, can go stale if the DOM changes, and are a legacy Puppeteer-style API — avoid for new tests.

Q11. What is XPath, and should you use it in Playwright? XPath is a query language for navigating XML/HTML trees. Playwright supports it (`page.locator('xpath=//button')`), but it's discouraged as a default choice because it's brittle against markup changes — reach for `role`/`label`/`text` locators first.

Q12. What is a CSS selector? A pattern (`#id`, `.class`, `div > button`) used to match HTML elements; useful when no semantic attribute exists, but less resilient than role-based locators.

Q13. How do you handle dynamic/changing elements? Anchor to stable attributes (`data-testid`, ARIA role, label) rather than auto-generated classes or DOM position, so tests don't break when styling or markup shifts.

Q14. How does Playwright handle Shadow DOM? Locators automatically pierce open Shadow DOM trees, so you can query into web components the same way as regular elements without extra syntax.

3. Waiting Strategies & Flaky Tests

Q15. What is auto-waiting in Playwright? Before an action (`click`, `fill`, `hover`), Playwright automatically waits for the target element to be attached to the DOM, visible, stable

(not animating), enabled, and able to receive events — removing most need for manual waits.

Q16. What wait strategies/APIs does Playwright expose?

```
await page.locator('#el').waitFor({ state: 'visible' });
await page.waitForLoadState('networkidle');
await page.waitForSelector('#el');
await page.waitForFunction(() => document.querySelectorAll('.item').length > 5);
await page.waitForURL('**/dashboard');
await page.waitForResponse('**/api/users');
await page.waitForEvent('download');
await page.waitForTimeout(3000); // last resort only
```

Q17. What is the default action timeout, and how do you configure it? 30 seconds by default; override per action ({ timeout: 5000 }) or globally with page.setDefaultTimeout(60000).

Q18. What causes flaky tests? Hard-coded sleeps, unstable/brittle selectors, timing/race conditions, shared test data, and environment inconsistencies.

Q19. How do you reduce flaky tests? Rely on auto-wait instead of fixed timeouts, use resilient role/label/text locators, add configured retries for genuinely unstable environments, keep test data and environments consistent, and prefer web-first assertions (expect(locator)...) which retry automatically.

Q20. What is “retry” in Playwright configuration? A config option (retries: N) that automatically reruns a failed test up to N times before marking it failed — useful for absorbing environment flakiness, not a substitute for fixing root causes.

4. Actions & UI Interactions

Q21. How do you handle dropdowns?

```
await page.locator('select#country').selectOption('US');
```

Q22. How do you handle alerts, prompts, and confirmations?

```
page.on('dialog', async dialog => {
  if (dialog.type() === 'prompt') await dialog.accept('User Input');
  else if (dialog.message().includes('Are you sure?')) await dialog.accept();
  else await dialog.dismiss();
});
// One-time handling:
page.once('dialog', dialog => dialog.accept());
```

Q23. How do you handle file upload and download?

```
// Upload
await page.locator('input[type="file"]').setInputFiles('file.pdf');
// Upload from an in-memory buffer
await page.locator('input[type="file"]').setInputFiles({
  name: 'report.pdf', mimeType: 'application/pdf', buffer: Buffer.from('content')
});
// Download
const downloadPromise = page.waitForEvent('download');
await page.locator('#download-link').click();
const download = await downloadPromise;
await download.saveAs('/path/file.pdf');
```

Q24. How do you handle frames/iframes?

```
const frame = page.frameLocator('#frameId');
await frame.locator('button').click();
```

frameLocator() is the recommended, auto-waiting way to reach into embedded frames (including nested ones).

Q25. How do you handle new tabs/pages?

```
const newPagePromise = context.waitForEvent('page');
await page.locator('a[target="_blank"]').click();
const newPage = await newPagePromise;
await newPage.waitForLoadState();
```

Q26. How do you perform low-level keyboard/mouse actions?

```
await page.keyboard.press('Enter');
await page.mouse.move(100, 200);
await page.mouse.down();
await page.mouse.up();
```

Useful for drag-and-drop by coordinates or custom gesture sequences that a simple click() can't express.

Q27. How would you test drag-and-drop?

```
await page.locator('[data-task="task-1"]').dragTo(page.locator('[data-column="in-progress"]'));
```

For components that ignore the native HTML5 drag events, fall back to manual mouse.move/down/up sequences using bounding boxes.

5. Browser Contexts, Sessions & Authentication

Q28. What is a browser context, and why use multiple ones? An isolated session (own cookies, cache, local storage) inside one browser instance. Multiple contexts let you simulate multiple independent users, test different permission states, or isolate test data — without launching multiple browsers.

```
const context1 = await browser.newContext();
const context2 = await browser.newContext();
```

Q29. How do you save and reuse authentication state (avoid logging in every test)?

```
// Save once after logging in
await context.storageState({ path: 'auth.json' });
// Reuse in later tests
const context = await browser.newContext({ storageState: 'auth.json' });
```

This can also be wired into a test.beforeEach, a custom fixture, or fetched via an API-based login for speed.

6. API Testing & Network Interception

Q30. What is API testing in Playwright, and how do you do it? Playwright's request context lets you call backend endpoints directly (no browser needed) — useful for setup/teardown

or pure API test suites.

```
const response = await request.post('/api/login', { data: { user, pass } });
expect(response.status()).toBe(200);
```

Q31. How do you validate API responses triggered by UI actions?

```
const response = await page.waitForResponse('**/api/login');
expect(response.status()).toBe(200);
```

You can assert status codes, response body, headers, and timing — combining UI and API validation in one test.

Q32. How do you mock or intercept network requests?

```
await page.route('**/api/users', route => route.fulfill({
  status: 200, contentType: 'application/json',
  body: JSON.stringify([{ id: 1, name: 'John' }])
}));
await page.route('**/*.{png,jpg,jpeg}', route => route.abort()); // block assets
```

Q33. What is an assertion, and how do web-first assertions differ from plain ones? An assertion verifies actual vs. expected results. Web-first assertions like `expect(locator).toBeVisible()` auto-retry until the condition is true or timeout — unlike `expect(await locator.textContent()).toBe(...)`, which checks a single snapshot with no retry.

7. Test Organization: POM, Fixtures & Data-Driven Testing

Q34. What is the Page Object Model (POM)? A design pattern that encapsulates a page's locators and actions inside a class, keeping test files focused on behavior rather than selectors.

```
class LoginPage {
  constructor(page) {
    this.page = page;
    this.username = page.getByLabel('Username');
    this.password = page.getByLabel('Password');
    this.loginBtn = page.getByRole('button', { name: 'Login' });
  }
  async login(user, pass) {
    await this.username.fill(user);
    await this.password.fill(pass);
    await this.loginBtn.click();
  }
}
```

Q35. What are fixtures, and how do you write a custom one? Reusable setup/teardown units injected into tests by name. Custom fixtures extend the base test object:

```
exports.test = base.test.extend({
  authenticatedPage: async ({ page }, use) => {
    await page.goto('/login');
    await page.fill('#username', 'testuser');
    await page.click('#login');
    await use(page);
  }
});
```

```
}  
});
```

Q36. What do `test.describe`, `test.beforeEach`, and `test.afterEach` do? `test.describe` groups related tests; `beforeEach` runs setup before every test in scope; `afterEach` runs cleanup after every test — commonly used for login, test-data seeding, and teardown.

Q37. How do you skip a test or isolate a single test in code?

```
test.skip('not ready yet', async ({ page }) => { /* ... */ });  
test.skip(condition, 'reason for skipping'); // conditional skip  
test.only('focus on this one', async ({ page }) => { /* ... */ }); // runs only this test in th
```

`test.only()` is handy while iterating locally but shouldn't be committed, since it silently narrows CI runs to one test.

Q38. What is data-driven testing, and how do you read test data from JSON? Running the same test logic against multiple data sets, e.g. import a JSON file and for (const data of testData) { test(case \${data.id}, ...) }.

8. Parallel Execution, Sharding & CI/CD

Q39. How do you run tests in parallel? Configure workers in `playwright.config.js`; Playwright's test runner parallelizes across worker processes with fully isolated browser contexts by default (`fullyParallel: true`).

Q40. How do you force specific tests to run serially?

```
test.describe.serial('checkout flow', () => {  
  test('add to cart', async ({ page }) => {});  
  test('checkout', async ({ page }) => {});  
});
```

Use when later tests depend on state created by earlier ones.

Q41. What is test sharding? Splitting the full test suite across multiple machines/CI jobs:

```
npx playwright test --shard=1/3
```

Q42. How do you integrate Playwright with CI/CD (e.g., GitHub Actions)? Install dependencies, run `npx playwright test headless`, and store the HTML report/traces as build artifacts. Add a YAML workflow that triggers on push/PR. Playwright also runs on Jenkins and GitLab CI.

Q43. What is CI/CD, in plain terms? Continuous Integration / Continuous Deployment — pipelines that automatically build, test, and (optionally) deploy code on every change.

9. CLI Commands & Test Execution Reference

(New section — a copy-ready cheat sheet of the commands you'll actually type day to day.)

Q44. How do you run the entire test suite?

```
npx playwright test
```

Q45. How do you run tests in headed mode, or scoped to one browser/project?

```
npx playwright test --headed
npx playwright test --project=chromium --headed
```

Note: projects must be defined in `playwright.config.ts` first.

Q46. How do you run a single spec file or a single test by name?

```
npx playwright test <file_name.ts> --project=chromium
npx playwright test -g "<test-name>"
```

-g matches on the test title via pattern, so keep test names unique enough to target safely.

Q47. How do you record a test instead of writing one from scratch?

```
npx playwright codegen
```

Opens a browser that records your clicks/typing and generates locator-based code — a fast way to scaffold a test or discover good selectors for an unfamiliar page.

Q48. How do you view the HTML report after a run?

```
npx playwright show-report
```

Q49. How do you force a trace on every run instead of only on retry?

```
npx playwright test --project=chromium --trace on
```

Config default is usually trace: 'on-first-retry' (only capture a trace when a test fails and gets retried, keeping overhead low); switch to 'on' when you specifically need traces for every run, e.g. while investigating a flaky suite.

Q50. What does UI mode give you beyond a normal CLI run?

```
npx playwright test --ui
```

Benefits: run individual tests/projects from the config, inspect network calls and the full execution trace, view before/after hooks and logs, create/inspect locators like a browser inspector, see exactly where clicks/text input happened (highlighted on screen), and auto-watch mode that re-runs tests as code changes.

Q51. What does the Playwright VS Code Test Explorer extension add? It brings the same visual debugging into the editor: toggle “show browser,” pick which projects are visible, enable the trace viewer, keep the browser open after a run for exploration, and see execution time with quick navigation straight to failing steps — useful when you’d rather not leave your IDE to debug.

10. Debugging, Reporting & Tracing

Q52. What is Trace Viewer? A post-run debugging tool that replays a test’s execution with DOM snapshots, screenshots, network logs, and console output — invaluable for diagnosing CI-only failures.

```
npx playwright show-trace trace.zip
```

Q53. How do you configure screenshots, video, and trace capture?

```
use: {
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
}
```



```
    trace: 'on-first-retry',
  }
```

Q54. How do you generate an HTML report from config (vs. viewing it via CLI)?

```
reporter: [['html', { open: 'never' }]]
```

This controls report *generation* at config level; `npx playwright show-report` (Q48) is how you *open* it afterward.

Q55. What are the main ways to debug a failing Playwright test? - `npx playwright test --debug` — opens the interactive Inspector plus a browser window, letting you step through each action and inspect the page live; especially useful for picking locators and reproducing flaky steps. - `--headed / slowMo` — watch the run at human speed. - `page.pause()` — drop into the Inspector at a specific line. - Listening to `page.on('console')` / `page.on('pageerror')` for browser-side errors. - The VS Code extension's built-in breakpoints (Q51).

11. Visual Regression Testing

Q56. How do you do visual regression testing in Playwright?

```
await expect(page).toHaveScreenshot('homepage.png');
await expect(page).toHaveScreenshot('homepage.png', { maxDiffPixelRatio: 0.02 });
await expect(page).toHaveScreenshot('homepage.png', {
  mask: [page.locator('.timestamp')] // ignore dynamic content
});
```

Update baselines with `npx playwright test --update-snapshots`; combine with the `browser-Name` fixture for cross-browser visual checks.

12. Advanced / Often-Missed Features

(Added to fill gaps in the source material — commonly asked in 2026-era interviews.)

Q57. What are soft assertions? `expect.soft(locator).toHaveText(...)` records a failure but keeps the test running, so you can collect multiple assertion failures in one run instead of stopping at the first one.

Q58. What is `expect.poll()` / `toPass()` used for? They retry an arbitrary function or block until it stops throwing or a condition is met — useful for polling non-locator state (e.g., a value computed from multiple sources) the way web-first assertions do for elements.

Q59. How do you emulate mobile devices or viewport/geolocation?

```
const { devices } = require('@playwright/test');
test.use({ ...devices['iPhone 14'] });
```

Device descriptors set viewport, user agent, and touch support; you can also set geolocation, permissions, locale, and colorScheme per context.

Q60. How do you test accessibility with Playwright? Use role-based locators (`getByRole`) as a natural fit for a11y, plus the `@axe-core/playwright` integration to run automated accessibility audits against a page and assert on violations.

Q61. What is `test.step()`? Wraps a block of actions with a named label that shows up in reports and traces, making long tests easier to read and debug without changing behavior.

Q62. What are test tags/annotations for? `test('...', { tag: '@smoke' }, async ({ page }) => {})` lets you filter runs, e.g. `npx playwright test --grep @smoke`, useful for separating smoke/regression/slow suites.

Q63. What is global setup/teardown? `globalSetup/globalTeardown` in the config run once before/after the entire test run (e.g., seeding a database or performing a single login to generate `storageState`), separate from per-test `beforeEach/afterEach`.

Q64. Does Playwright support TypeScript? Yes, natively and with zero extra config — `.ts` test files work out of the box and give locator/API type-checking and autocomplete, which is a common reason teams choose it over Selenium.

Q65. What is component testing in Playwright? `@playwright/experimental-ct-
<framework>` (React/Vue/Svelte) mounts individual UI components in a real browser for isolated testing, bridging the gap between unit tests and full E2E tests.

13. Scenario-Based / Real-World Questions

Q66. How would you test an infinite-scroll table? Loop: scroll to the last row, wait for either the network response (`page.waitForResponse('**/api/users*')`) or a row-count change, and stop when the count no longer increases.

Q67. How would you test a multi-step form with validation and back-navigation? Fill each step, assert validation errors appear when required fields are empty, navigate back and confirm previously entered data persisted, then submit and assert the final API call's status and resulting URL.

Q68. How would you test a real-time chat app? Open two isolated browser contexts (simulating two users), have one send a message, and assert it appears in both contexts' message lists — optionally also asserting a typing indicator and testing reconnect behavior via `context.setOffline(true/false)`.

Q69. How would you automate a login flow that requires OTP? Enter credentials, wait for the OTP-triggering API call, retrieve the OTP from a test-environment source (mock endpoint, test inbox, or DB), submit it, and assert successful login — plus negative tests for invalid or expired OTPs.

Q70. How would you test file-based reporting/export features (e.g., “Download CSV”)? Trigger the action, await the download event, save it, and then parse the saved file's contents to assert the exported data matches expectations, rather than only checking that a download occurred.

14. General QA & SDLC Concepts

(Frequently paired with Playwright-specific questions in interviews.)

Q71. Smoke vs. Sanity vs. Regression testing? Smoke = quick check that critical paths work at all. Sanity = quick, focused validation of a specific change. Regression = broader check that existing functionality still works after changes.

Q72. Severity vs. Priority? Severity measures a bug's impact on the system; priority measures how urgently it needs fixing — a high-severity bug isn't always the highest priority (and vice versa).

Q73. What is the bug life cycle? The path a defect follows from being logged, triaged, assigned, fixed, retested, and closed (with variants like reopened or deferred along the way).

Q74. What is a test strategy? The high-level plan defining scope, approach, environments, tools, and risk areas for testing an application or release.

Q75. What is performance, load, and security testing? Performance = overall speed/responsiveness; load = behavior under expected/peak user volume; security = probing for vulnerabilities — distinct from functional testing but often part of a full QA strategy.

Q76. What is Agile, and what's a sprint? Agile is an iterative delivery approach; a sprint is a fixed, time-boxed development cycle (commonly 1–2 weeks) within it.

Q77. What are JIRA and Git used for? JIRA tracks issues, stories, and sprints; Git is the version control system used to manage and collaborate on source code, including test automation code.

Q78. How do you move from manual to automation testing? Build core programming fundamentals, pick a tool (e.g., Playwright), and practice by automating real project scenarios rather than only tutorials.

Q79. What makes someone “job-ready” for automation roles in 2026? Solid fundamentals, hands-on Playwright skills (locators, waits, fixtures, CI/CD), and comfort using AI-assisted tooling (codegen, AI-suggested locators/tests) as part of the workflow.

15. Best Practices & Common Pitfalls

Do: - Prefer `getByRole` / `getByLabel` / `getByText` over CSS/XPath. - Rely on auto-wait and web-first assertions instead of fixed sleeps. - Encapsulate page logic in POM classes; keep tests readable. - Parallelize (`fullyParallel: true`) and shard for large suites. - Capture trace/video/screenshot on-first-retry / only-on-failure. - Keep tests independent — no shared mutable state between them. - Use fixtures and `globalSetup` for setup, not copy-pasted boilerplate. - Use codegen and UI mode to speed up authoring and debugging, not as a substitute for understanding locators.

Avoid: - Fixed timeouts (`waitForTimeout`) as a primary strategy. - Element handles (`page.$`) over locators. - Brittle selectors tied to DOM position or auto-generated classes. - Test inter-dependence (test B silently depending on test A's side effects). - Hardcoded test data instead of fixtures/generators. - Over-mocking to the point real integration issues go undetected. - Skipping CI/CD integration — local-only test suites decay quickly. - Committing `test.only()` — it silently narrows what CI actually runs.

16. Suggested Study Approach

Given the material, here's how I'd approach prepping with it:

1. **Anchor on the “why” before memorizing syntax.** Interviewers usually probe *why* Playwright auto-waits, *why* locators beat element handles, *why* contexts are isolated — not just the API. Sections 1–3 above are the conceptual core; get those solid first.
2. **Build one small real project instead of only reading.** Pick 3–4 scenarios from Section 13 (form validation, file download, API+UI hybrid check) and actually write them. Interviewers can tell rehearsed answers from lived experience within a few follow-up questions.
3. **Memorize the CLI cheat sheet (Section 9) cold.** Commands like `--project`, `-g`, `--trace on`, and `--ui` come up constantly in practical/live-coding rounds — being fluent here signals real hands-on experience fast.

4. **Treat Section 12 (advanced/often-missed) as your differentiator.** Most candidates know auto-wait and POM; fewer can speak fluently about `test.step`, soft assertions, component testing, or global setup — these are easy points if you’re prepared.
5. **Use Section 14 sparingly and contextually.** These general QA/SDLC questions matter more for junior/mixed manual-automation roles; if you’re interviewing for a pure SDET/automation-engineer role, weight your prep toward Sections 1-12 instead.
6. **Rehearse scenario questions out loud, not just read them.** Section 13 questions are usually asked as “walk me through how you’d design this test” — practicing the spoken explanation matters more than the code itself.
7. **Keep a “gotchas” flashcard set** from the Best Practices section (Section 15) — these are common rapid-fire final-round questions.

If it’s useful, I can also turn this into a Word document (with a table of contents and print-friendly formatting) or a PDF — just say the word.